Chapter 18

Concurrency Control

- Concurrency Control Protocol
- Deadlock Handling

Concurrency Control Protocol

- Concurrency control protocols assure serializability of schedules.
- Testing a schedule for serializability *after* it has executed is a little too late!

  - **Lock-Based Protocols**
  - **Timestamp-Based Protocols**

Locks

Data items can be locked in two modes :

1. *exclusive (X) mode*. The Transaction which obtained an exclusive-mode lock on item Q can both read and write Q.
2. *shared (S) mode*. The Transaction which obtained a shared-mode lock on item Q can read but cannot write Q.

Lock-compatibility matrix:

	S	X
S	true	false
X	false	false

- A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions

- Any number of transactions can hold shared locks on an item, but if any transaction holds an exclusive on the item no other transaction may hold any lock on the item.

- If a lock cannot be granted, the requesting transaction is made to wait till all incompatible locks held by other transactions have been released. The lock is then granted.

Lock-Based Protocols

A **locking protocol** is a set of rules followed by all transactions while requesting and releasing locks.

T_2: **lock-S**(*A*);

read (*A*);

unlock(*A*);

lock-S(*B*);

read (*B*);

unlock(*B*);

display(*A*+*B*)

Locking as above is not sufficient to guarantee serializability — if *A* and *B* get updated in-between the read of *A* and *B*, the displayed sum would be wrong.

Pitfalls of Lock-Based Protocols

Locking protocols restrict the set of possible schedules.

T_3	T_4
lock-X(B)	
read(B)	
$B := B - 50$	
write(B)	
	lock-S(A)
	read(A)
	lock-S(B)
lock-X(A)	

deadlock.

A transaction may be waiting for an X-lock on an item, while a sequence of other transactions request and are granted an S-lock on the same item.

Starvation

- **Starvation** is also possible if concurrency control manager is badly designed. For example:
  - A transaction may be waiting for an X-lock on an item, while a sequence of other transactions request and are granted an S-lock on the same item.
  - The same transaction is repeatedly rolled back due to deadlocks.
- Concurrency control manager can be designed to prevent starvation.

The Two-Phase Locking Protocol

Phase 1: Growing Phase

- ★transaction may obtain locks
- ★transaction may not release locks

Phase 2: Shrinking Phase

- ★transaction may release locks
- ★transaction may not obtain locks

The protocol assures **conflict serializability**. It can be proved that the transactions can be serialized in the order of their **lock points**(the point where a transaction acquired its final lock,the end of its growing phase).

- Two-phase locking *does not* ensure freedom from deadlocks

T_5	T_6	T_7
lock-X(A)		
read(A)		
lock-S(B)		
read(B)		
write(A)		
unlock(A)		
	lock-X(A)	
	read(A)	
	write(A)	
	unlock(A)	
		lock-S(A)
		read(A)

Cascading roll-back is possible under two-phase locking.

- Cascading roll-back is possible under two-phase locking. To avoid this, follow a modified protocol called **strict two-phase locking**. Here a transaction must hold all its exclusive locks till it commits/aborts.
- **Rigorous two-phase locking** is even stricter: here *all* locks are held till commit/abort. In this protocol transactions can be serialized in the order in which they commit.

Lock Conversions

Two-phase locking with lock conversions:

- First Phase:
  - ★ can acquire a **lock-S** on item
  - ★ can acquire a **lock-X** on item
  - ★ can convert a **lock-S** to a **lock-X** (**upgrade**)
- Second Phase:
  - ★ can release a **lock-S**
  - ★ can release a **lock-X**
  - ★ can convert a **lock-X** to a **lock-S** (**downgrade**)

This protocol assures serializability.

T_8	T_9
lock-S (a_1)	
	lock-S (a_1)
lock-S (a_2)	
	lock-S (a_2)
lock-S (a_3)	
lock-S (a_4)	
	unlock(a_1)
	unlock(a_2)
lock-S (a_n)	
upgrade (a_1)	

- The operation **read**(D) is processed as:

  if T_i has a lock on D
  then
  read(D)
  else
  begin
  if necessary wait until no other transaction has a **lock-X** on D
  grant T_i a **lock-S** on D;
  read(D)
  end

write(*D*) is processed as:

if T_i has a **lock-X** on *D*
then
write(*D*)
else
begin
if necessary wait until no other trans. has any lock on *D*,
if T_i has a **lock-S** on *D*
then
upgrade lock on *D* to **lock-X**
else
grant T_i a **lock-X** on *D*
write(*D*)
end;

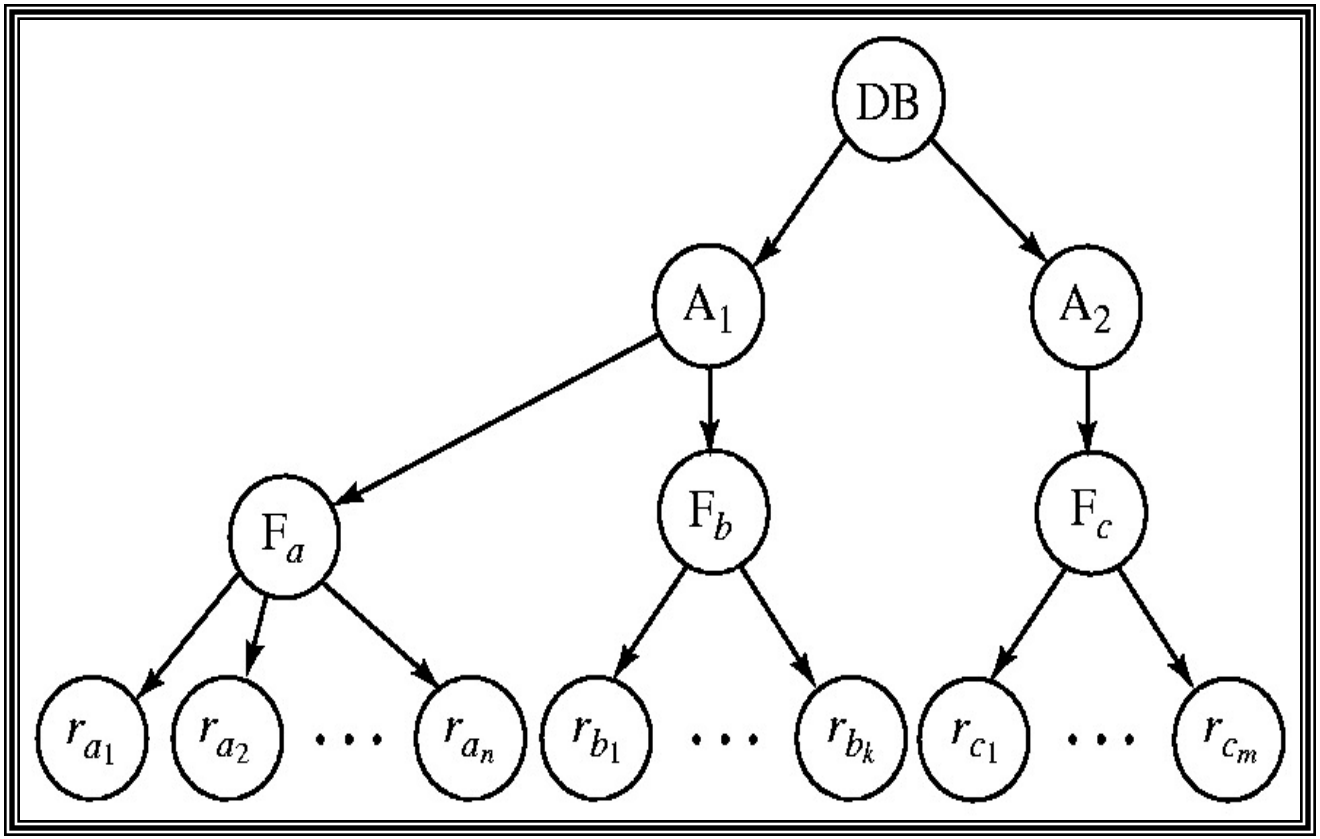
- All locks are released after commit or abort

T1	T2
Lock-X(A) Read(A) Lock-S(B) Read(B) Write(A) Unlock(A)	
	Lock-X(A) Read(A) A=:A+1 Write(A) Unlock(A)
Unlock(B)	

T1	T2
Lock-X(a1) Read(a1) A1:=a1+1 Write(a1) Unlock(a1)	
	Lock-S(a1) Read(a1)
Lock-X(a2) Read(a2) Unlock(a2)	
	Unlock(a1)

Multiple Granularity

- Allow data items to be of various sizes and define a hierarchy of data granularities, where the small granularities are nested within larger ones
- Can be represented graphically as a tree (but don't confuse with tree-locking protocol)
- When a transaction locks a node in the tree *explicitly*, it *implicitly* locks all the node's descendents in the same mode.
- Granularity of locking (level in tree where locking is done):
  - ★*fine granularity* (lower in tree): high concurrency, high locking overhead
  - ★*coarse granularity* (higher in tree): low locking overhead, low concurrency



The highest level in the example hierarchy is the entire database.

The levels below are of type *area*, *file* and *record* in that order.

Intention Lock Modes

There are three additional lock modes with multiple granularity:

- ★ ***intention-shared*** (IS): indicates explicit locking at a lower level of the tree but only with shared locks.
- ★ ***intention-exclusive*** (IX): indicates explicit locking at a lower level with exclusive or shared locks
- ★ ***shared and intention-exclusive*** (SIX): the subtree rooted by that node is locked explicitly in shared mode and explicit locking is being done at a lower level with exclusive-mode locks.

▪ intention locks allow a higher level node to be locked in S or X mode without having to check all descendent nodes.

Compatibility Matrix

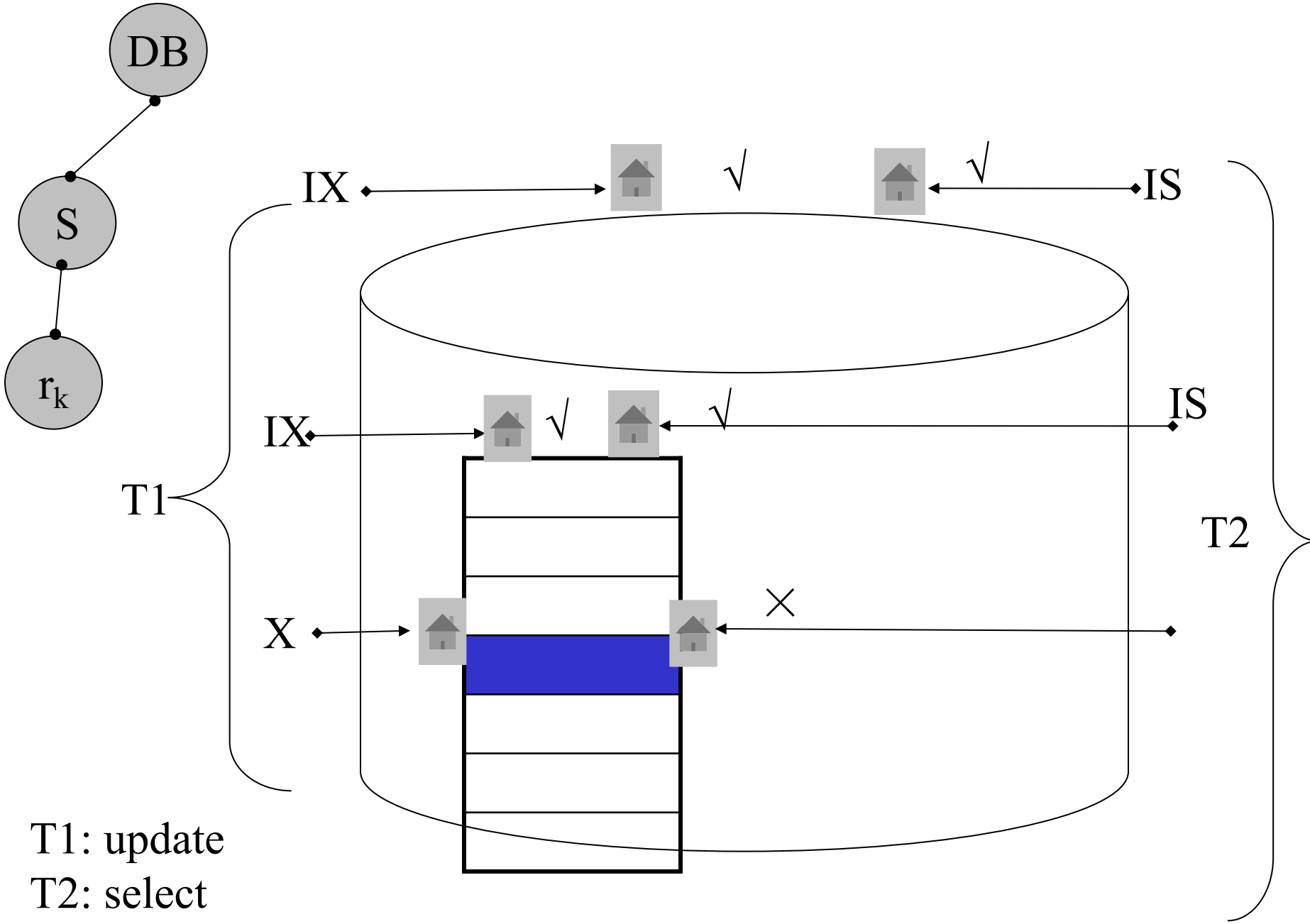
	IS	IX	S	S IX	X
IS	✓	✓	✓	✓	×
IX	✓	✓	×	×	×
S	✓	×	✓	×	×
S IX	✓	×	×	×	×
X	×	×	×	×	×

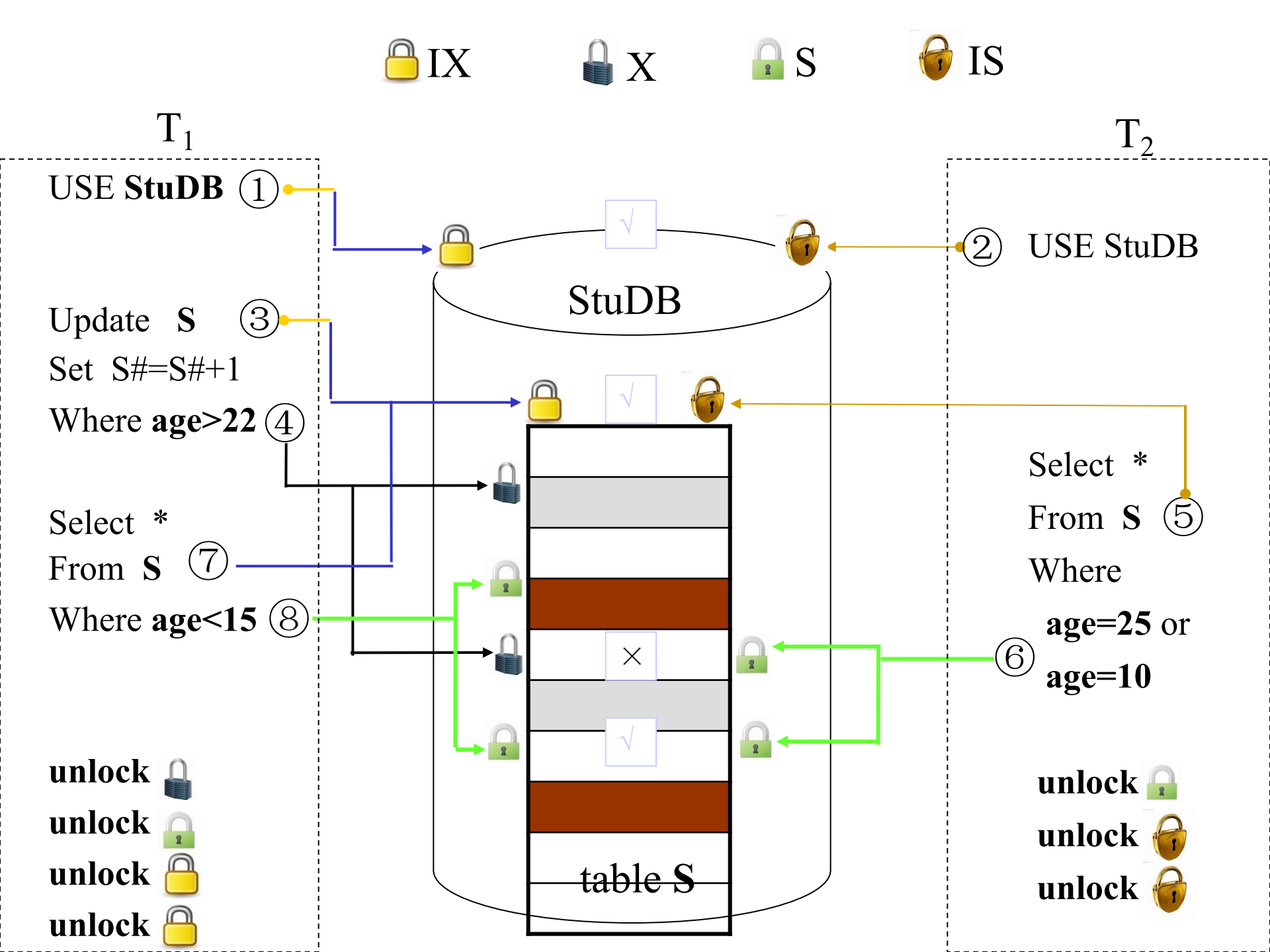
Locking Scheme

Transaction T_i can lock a node Q, using the following rules:

1. The lock compatibility matrix must be observed.
2. The root of the tree must be locked first, and may be locked in any mode, locks are acquired in root-to-leaf order.
3. A node Q can be locked by T_i in S or IS mode only if the parent of Q is currently locked by T_i in either IX or IS mode.
4. A node Q can be locked by T_i in X, SIX, or IX mode only if the parent of Q is currently locked by T_i in either IX or SIX mode.
5. T_i can lock a node only if it has not previously unlocked any node (that is, T_i is two-phase).
6. T_i can unlock a node Q only if none of the children of Q are currently locked by T_i.

Locks are released in leaf-to-root order.


DB
S
r_k
IX
√
√
IS
IX
√
√
IS
T1
T2
X
×
T1: update
T2: select


IX
X
S
IS
T1
T2
USE StuDB ①
② USE StuDB
StuDB
Update S ③
Set S#=S#+1
Where age>22 ④
Select *
From S ⑤
Where
age=25 or
age=10
⑥
Select *
From S ⑦
Where age<15 ⑧
table S
unlock
unlock
unlock
unlock
unlock
unlock
unlock

Deadlock Handling

System is **deadlocked** if there is a set of transactions such that every transaction in the set is waiting for another transaction in the set,and none of the transactions can ever proceed with its normal executions.

T_1	T_2
lock-X on X	
write (X)	
	lock-X on Y
	write (X)
wait for **lock-X** on Y	wait for **lock-X** on X

Deadlock Prevention Strategies

Deadlock prevention protocols ensure that the system will *never* enter into a deadlock state.

★Require that each transaction locks all its data items before it begins execution (predeclaration).

★Impose partial ordering of all data items and require that a transaction can lock data items only in the order specified by the partial order (graph-based protocol).

★Timeout-Based Schemes :

a transaction waits for a lock only for a specified amount of time. After that, the wait times out and the transaction is rolled back.

Following schemes use transaction timestamps for the sake of deadlock prevention alone:

- **wait-die** scheme

If TS(Ti) < TS(Tj) then Ti Waits
Else
RollBack Ti
Restart Ti with the same TS(Ti)
Endif

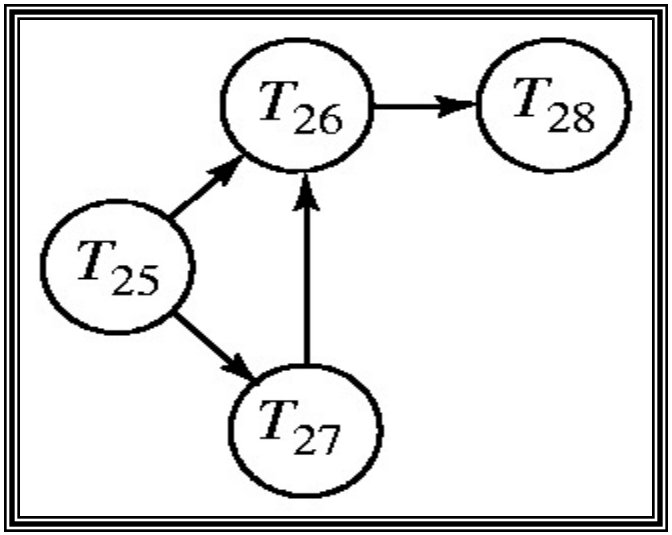
- **wound-wait** scheme

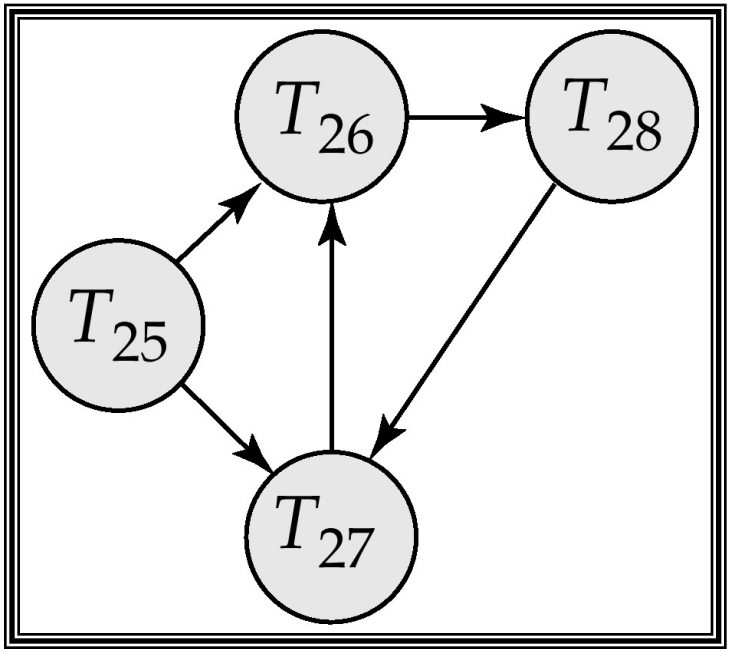
If TS(Ti) > TS(Tj) then Ti Waits
Else
RollBack Tj
Restart Tj with the same TS(Tj)
Endif

- **wait-die** scheme — non-preemptive
  - ★older transaction may wait for younger one to release data item. Younger transactions never wait for older ones; they are rolled back instead.
  - ★a transaction may die several times before acquiring needed data item
- **wound-wait** scheme — preemptive
  - ★older transaction *wounds* (forces rollback) of younger transaction instead of waiting for it. Younger transactions may wait for older ones.
  - ★may be fewer rollbacks than *wait-die* scheme.

Deadlock Detection

- Deadlocks can be described as a *wait-for graph*, which consists of a pair $G = (V,E)$,
  - V is a set of vertices (all the transactions in the system)
  - E is a set of edges; each element is an ordered pair $T_i \rightarrow T_j$.
- If $T_i \rightarrow T_j$ is in E, then there is a directed edge from T_i to T_j, implying that T_i is waiting for T_j to release a data item.
- The system is in a deadlock state if and only if the wait-for graph has a cycle. Must invoke a deadlock-detection algorithm periodically to look for cycles.


T_{26}
T_{28}
T_{25}
T_{27}


T_{26}
T_{28}
T_{25}
T_{27}

Deadlock Recovery

When deadlock is detected :

- ★Some transaction will have to rolled back (made a victim) to break deadlock. Select that transaction as victim that will incur minimum cost.
- ★Rollback -- determine how far to roll back transaction
  - ➢Total rollback: Abort the transaction and then restart it.
  - ➢More effective to roll back transaction only as far as necessary to break deadlock.
- ★Starvation happens if same transaction is always chosen as victim. Include the number of rollbacks in the cost factor to avoid starvation

Insert and Delete Operations

If two-phase locking is used :

- ★A **delete** operation may be performed only if the transaction deleting the tuple has an exclusive lock on the tuple to be deleted.
- ★A transaction that inserts a new tuple into the database is given an X-mode lock on the tuple

If the timestamp-ordering protocol is used:

- ★A **delete** operation: a test similar to that for a write must be performed;
- ★Transaction Ti Performs insert(Q),the values R-timestamp(Q) and W-timestamp(Q) are set to TS(Ti).

T1	T2	T3
		Lock-S(X)
		Read(X)
Lock-X(Y)		
Read(Y)		
Y:=Y*5		
Write(Y)		
	Lock_X(Z)	
	Read(Z)	
	Z:=Z/2	
	Write(Z)	
		Lock_S(Y)
Lock_S(Z)		
	Lock_X(X)	
		Read (Y)
Read(Z)		
	Read(X)	
Unlock (Y)		
	X:=X-50	
	Write(X)	
		Unlock (X)
	Unlock (Z)	
	Unlock (X)	
		Unlock(Y)
Unlock (Z)		

T_1	T_2	T_3	T_4
Lock_X(A) Read(A)			
	Lock_X(B)		
A:=A+10			
	Read(B)		
		Lock_X(C) Read(C)	
Write(A)			
	B:=B+50 Write(B)		
		C:=C+30	
	Unlock(B) commit		
Lock_X(B) Unlock(A) Read(B)			
		Lock_X(A) Unlock(C) Read(A)	
			Lock_X(C) Read(C)
B:=B+10 Write(B)			
		A:=A+100 Write(A)	
			C:=C*100
Unlock(B) Commit			
		Unlock(A) Commit	
			Unlock(C) Commit

(1) Is S conflict serializable? and why?

(2) Is S a cascading or cascadeless schedule? and why?

(3) Is S obey the two-phase locking protocol? and why?

(4) Is S obey the rigorous two-phase locking protocol? and why?